<HLS Programming with FPGAs>

AWS F1 DRAM Architecture and Optimizations

Choi, Young-kyu

Recap: Hello World Project

- Vector add kernel:

```
extern "C" {

void vadd(hls::vector<unsigned int, 16>* 
          hls::vector<unsigned int, 16>* 
          hls::vector<unsigned int, 16>* 
          int size) {
#pragma HLS INTERFACE m_axi port = in1 bu
#pragma HLS INTERFACE m_axi port = in2 bu
#pragma HLS INTERFACE m_axi port = out bu

    static hls::stream<hls::vector<unsigned int, 16> > in1_stream("input_stream_1");
    static hls::stream<hls::vector<unsigned int, 16> > in2_stream("input_stream_2");
    static hls::stream<hls::vector<unsigned int, 16> > out_stream("output_stream");

    assert(size % 16 == 0);
    int vSize = size / 16;
#pragma HLS dataflow

    load_input(in1, in1_stream, vSize);
    load_input(in2, in2_stream, vSize);
    compute_add(in1_stream, in2_stream, out_stre
    store_result(out, out_stream, vSize);
}
}
```

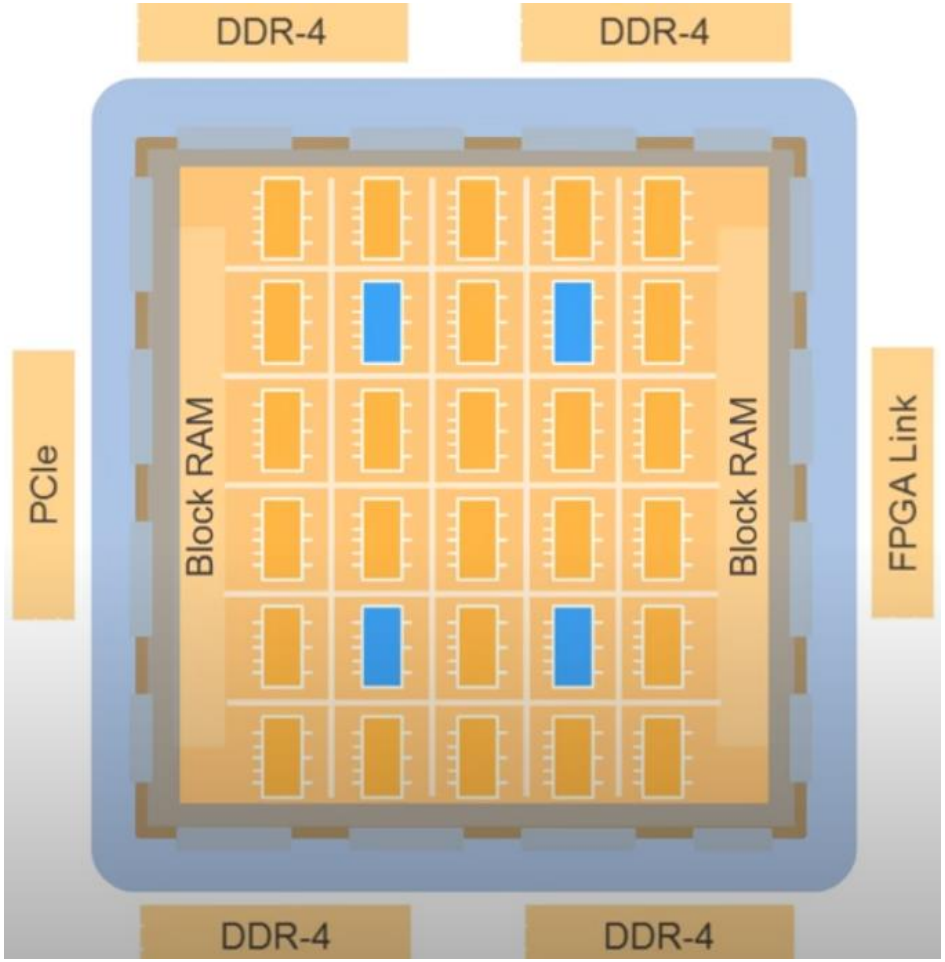
```
static void load_input(hls::vector<unsigned int, 16>* in,
                       hls::stream<hls::vector<unsigned int, 16> >& inStream,
                       int vSize) {
mem_rd:
    for (int i = 0; i < vSize; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        inStream << in[i];
    }
}
```

```
static void compute_add(hls::stream<hls::vector<unsigned int, 16> >& in1_stream,
                        hls::stream<hls::vector<unsigned int, 16> >& in2_stream,
                        hls::stream<hls::vector<unsigned int, 16> >& out_stream,
                        int vSize) {
execute:
    for (int i = 0; i < vSize; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        out_stream << (in1_stream.read() + in2_stream.read());
    }
}
```

Learning Objectives

- Explain the DRAM architecture of AWS F1
- Understand the reason for the low effective bandwidth in vector add application
- Optimize the DRAM access architecture with Vitis & Vitis HLS

DRAM Channels in AWS F1



David Pellerin, Deep Dive on Amazon EC2 F1 Instance, https://youtu.be/R_Wxc8y7lb0

External Memory Interface in AWS F1

- Interface between kernel and external memory (DRAM)
  - Interface : AXI bus**
  - Clock frequency: 250MHz
  - Data bitwidth : 512b
  - Number of DRAM channels : 4
  - Ideal DRAM BW: 4 * 0.250GHz * 512b/8 = 64 GB/s

** Xilinx, UG761, AXI Reference Guide

Motivating Example : Vector Add

- Suppose we used the following HLS code for vector add:

```
void vadd(unsigned int* in1,
          unsigned int* in2,
          unsigned int* out,
          int size) {

    for (int i = 0; i < size; i++) {
#pragma HLS PIPELINE II=1
        out[i] = in1[i] + in2[i];
    }
}
```

- What is the effective DRAM bandwidth of this baseline vector add code?

Motivating Example : Vector Add

- Experimental result on AWS F1 (on-board test)
  - vector add length (DATA_SIZE) : 256M
  - Time taken : 29.2 sec → What is effective DRAM BW?
- Assumption:
  - Vector add kernel spends most of time transferring data (memory-bound application)
- Effective BW = size of data in bytes / time_taken
  = 3 * 4B * DATA_SIZE / time_taken
  = 0.110 GB/s → Why low BW?
  <<<< 64 GB/s ideal DRAM BW

Problem 1 : Narrow Bus Data Bitwidth

- Physically available bus data bitwidth : 512b
- Synthesized bus data bitwidth : 32b
  - Kernel code :

```
void vadd(unsigned int* in1,
          unsigned int* in2,
          unsigned int* out,
          int size) {
```

  - Waveform (from co-sim or HW emu) :

Solution 1 - 1 : Wide data type

- Step 1) Use ap_uint<512> for top kernel function argument:
  - Code:

```
void vadd(ap_uint<512>* in1,
          ap_uint<512>* in2,
          ap_uint<512>* out,
          int size) {
```

- Step 2) Divide ap_uint<512> into original data type
  - Range operator: `ap_(u)int::operator ()(unsigned Hi, unsigned Lo )`
  - Code:

```
 ap_uint<512> in1_temp = in1[i];
ap_uint<32> in1_data0 = in1_temp( 31,  0);
ap_uint<32> in1_data1 = in1_temp( 63, 32);
ap_uint<32> in1_data2 = in1_temp( 95, 64);
```

……

  - Do the same for in2 argument as well

Solution 1 - 1 : Wide data type

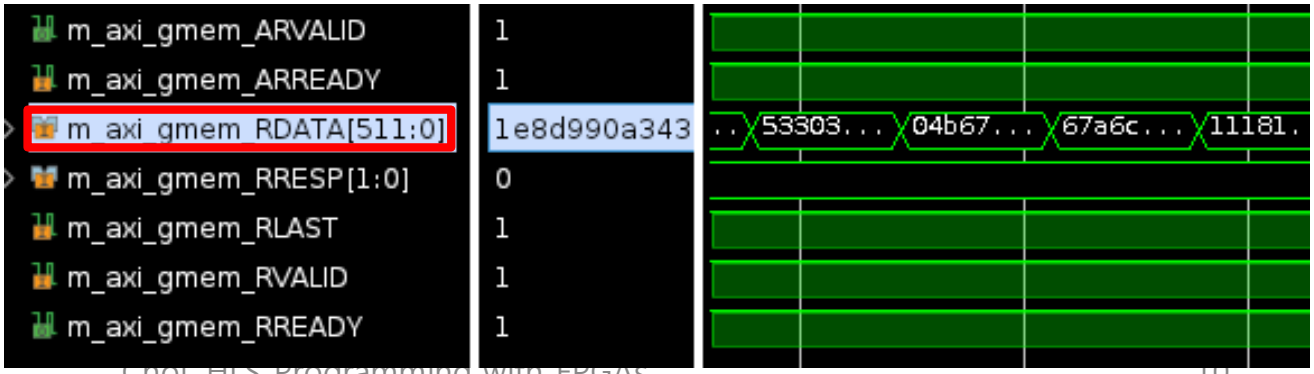
- Step 3) Perform addition in original data type
  - Code:

```
ap_uint<32> out_data0  = in1_data0  + in2_data0 ;
ap_uint<32> out_data1  = in1_data1  + in2_data1 ;
ap_uint<32> out_data2  = in1_data2  + in2_data2 ;
```

- Step 4) Concatenate result into ap_uint<512> type
  - Concatenation operator: `ap_(u)int::operator , (ap_(u)int high, ap_(u)int low)`
    - Code:

```
ap_uint<512> out_temp = (out_data15, out_data14, ..., out_data0);
out[i] = out_temp;
```

- Waveform:



Solution 1 - 2 : Vector data type

- Vitis HLS supports vector type (hls::vector)
- Recently added feature
- If using the base type of unsigned int, use a vector of 512b/32b = 16 elements
- Code:

```
void vadd(hls::vector<unsigned int, 16>* in1,
          hls::vector<unsigned int, 16>* in2,
          hls::vector<unsigned int, 16>* out,
          int size) {
```

```
    int vSize = size / 16;
    for (int i = 0; i < vSize; i++) {
#pragma HLS PIPELINE II=1
        out[i] = in1[i] + in2[i];
    }
```

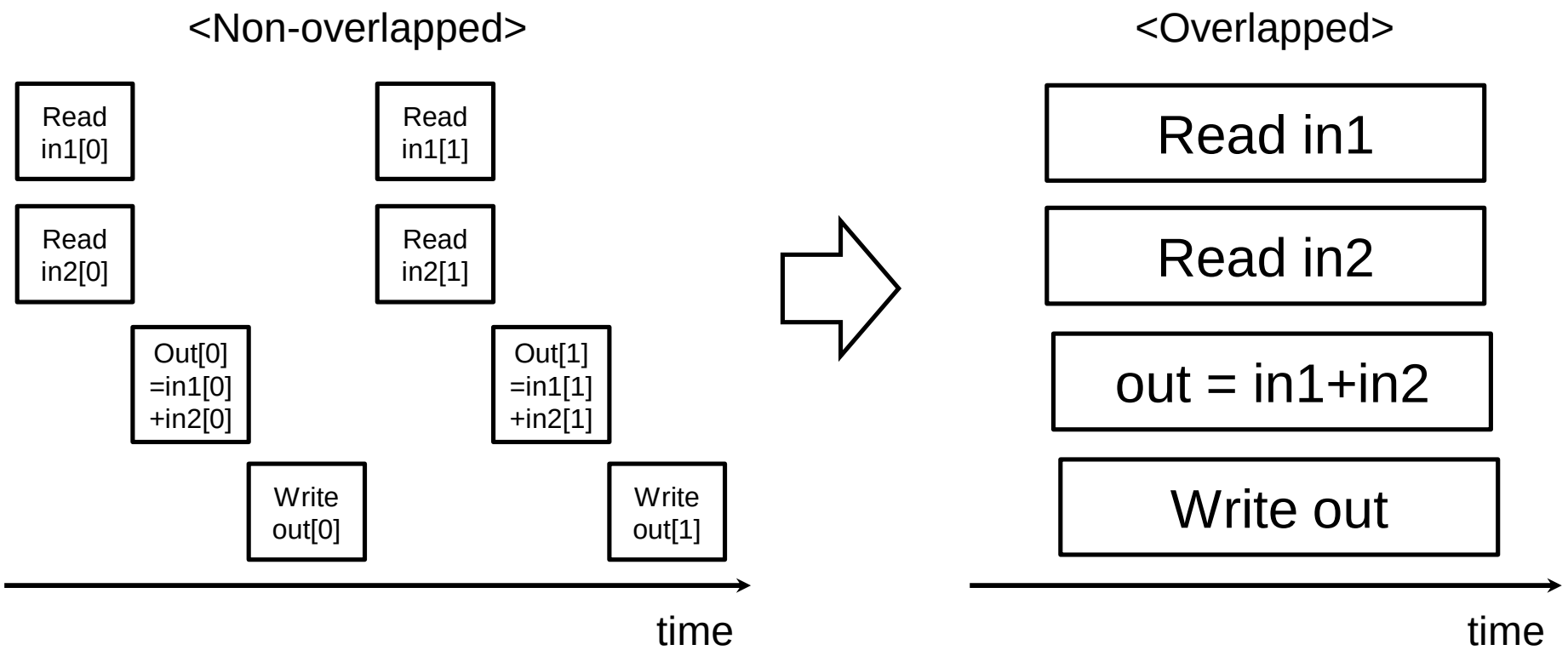
: vector type add of 16 elements per iteration

(old) Problem 2 : Computation-Memory Overlapping

- Code:

```
    for (int i = 0; i < size; i++) {
#pragma HLS PIPELINE II=1
        out[i] = in1[i] + in2[i];
    }
```

- Timing diagram:



(old) Solution 2 : Dataflow Optimization

- Dataflow optimization:
  - Overlaps the execution of functions (even if there is dependency between them)
- Compiler directive to enable dataflow optimization:
  - **#pragma HLS dataflow**
- Communication interface between the functions should be FIFO
  - Acts as data buffers
  - Instantiated using **hls::stream<base type>**

(old) Solution 2 : Dataflow Optimization

- vadd.cpp of hello_world project:

```
void vadd(hls::vector<unsigned int, 16>* in1,
          hls::vector<unsigned int, 16>* in2,
          hls::vector<unsigned int, 16>* out,
          int size) {
#pragma HLS INTERFACE m_axi port = in1 bundle = gmem0
#pragma HLS INTERFACE m_axi port = in2 bundle = gmem1
#pragma HLS INTERFACE m_axi port = out bundle = gmem0

    static hls::stream<hls::vector<unsigned int, 16> > in1_stream("input_stream_1");
    static hls::stream<hls::vector<unsigned int, 16> > in2_stream("input_stream_2");
    static hls::stream<hls::vector<unsigned int, 16> > out_stream("output_stream");

    assert(size % 16 == 0);
    int vSize = size / 16;
#pragma HLS dataflow

    load_input(in1, in1_stream, vSize);
    load_input(in2, in2_stream, vSize);
    compute_add(in1_stream, in2_stream, out_stream, vSize);
    store_result(out, out_stream, vSize);
}
```

FIFO interface between functions

Executes in parallel

(old) Solution 2 : Dataflow Optimization

- Computation-memory overlapping is automated in the recent Vitis HLS
  - Explicit dataflow optimization is no longer necessary for computation-memory overlapping
  - But dataflow coding style is still a good practice, since Vitis HLS sometimes misses function-level parallelization opportunity

Performance after Applying Solution 1 & 2

- Execution time for 256M elements: 0.236 sec
- Effective DRAM BW = 13.6 GB/s

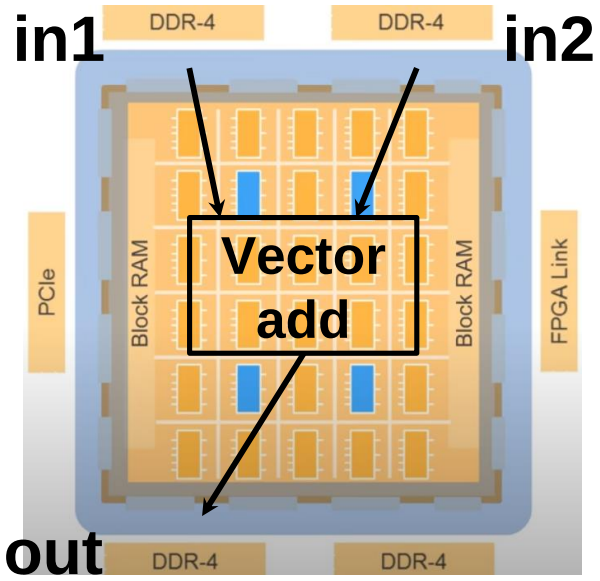
  >> 0.110GB/s (before optimization)

  < 64 GB/s (ideal DRAM BW)

  : some room for improvement

Problem 3 : Uses a Single DRAM channel

- All top function arguments (and corresponding OpenCL buffers) are assigned to a single DRAM channel by default
- Cannot utilize the full BW unless data are distributed among multiple DRAM channels
- Example allocation for higher effective DRAM BW:



Solution 3 : Use Multiple AXI Masters & Multiple DRAM Channels

- Goal :
  - Map top function arguments (in1, in2, out) to separate DRAM channels on AWS F1 (DDR[0], DDR[1], DDR[2])

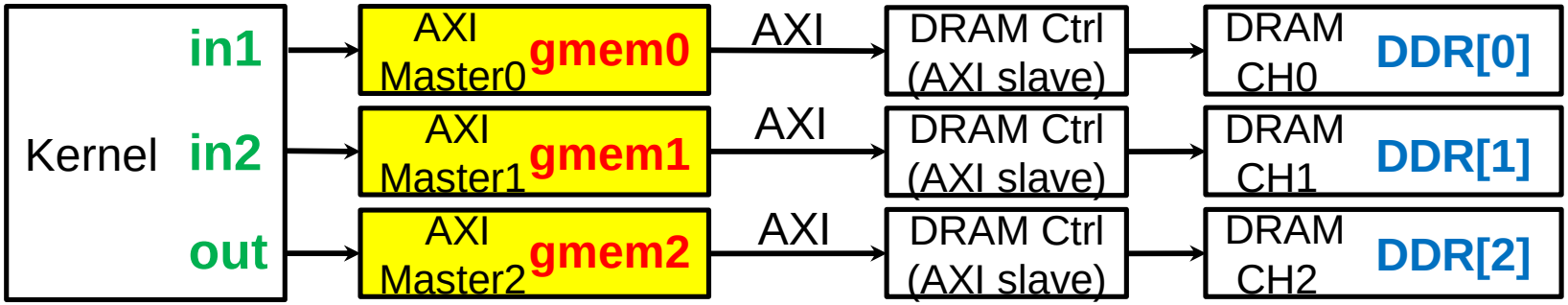
- Edit Makefile:

```
VPP_FLAGS += -t $(TARGET) --platform $(DEVICE) --save-temps
ifneq ($(TARGET), hw)
        VPP_FLAGS += -g
endif

VPP_FLAGS += --connectivity.sp vadd_1.in1:DDR[0]
VPP_FLAGS += --connectivity.sp vadd_1.in2:DDR[1]
VPP_FLAGS += --connectivity.sp vadd_1.out:DDR[2]
```

Solution 3 : Use Multiple AXI Masters & Multiple DRAM Channels

- Edit kernel file (vadd.cpp):
  - Bundle: Corresponds to an independent AXI master
  - Kernels use bundles to access DRAM channels



  - Should use separate bundle for parallel access
  - Modified code:

```
void vadd(hls::vector<unsigned int, 16>* in1,
          hls::vector<unsigned int, 16>* in2,
          hls::vector<unsigned int, 16>* out,
          int size) {
#pragma HLS INTERFACE m_axi port = in1 bundle = gmem0
#pragma HLS INTERFACE m_axi port = in2 bundle = gmem1
//#pragma HLS INTERFACE m_axi port = out bundle = gmem0
#pragma HLS INTERFACE m_axi port = out bundle = gmem2
```

Performance after Applying Solution 3

- Execution time for 256M elements: 0.082 sec
- Effective DRAM BW = 39.2 GB/s

  < 64 *3/4 (=48) GB/s ideal DRAM BW of 3 channels

- Comparison with baseline implementation

	On-board Eff BW
Baseline	0.110 GB/s
+ Vector optimized	13.6 GB/s
+ Multiple AXIs & DRAM CHs	39.2 GB/s

(*Dataflow optimization not effective for this kernel in Vitis 21.1)

Caution when Applying Solution 3

1. Using additional bundle and DRAM channels require additional FPGA resources (LUT, FF, BRAM, etc)
2. Needs to consider other kernels running in parallel
   - Overall performance might be better when independent DRAM channels are assigned to other memory-bounded kernels
   - Try to pass data directly to other kernels whenever possible (chaining) without consuming a DRAM channel

Further Readings

- Choi, Y. K., Cong, J., Fang, Z., Hao, Y., Reinman, G., & Wei, P. (2019). In-depth analysis on microarchitectures of modern heterogeneous CPU-FPGA platforms. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, *12*(1), 1-20.

Summary

- AWS F1 is composed of 4 independent DRAM channels
- Use wide uint data type or vector type to utilize 512b data bitwidth of AXI bus
- Function execution can be overlapped with dataflow optimization
- DRAM access modules may run independently when mapped to different bundles and DRAM channels